

Web VAPT Section 04 - Vulnerability Scanning and Fingerprinting

Professional documentation for Nikto and WhatWeb scan interpretation, misconfiguration review, and validation planning.

Enterprise Security Assessment Lab

PROJECT AREA	Web Application Vulnerability Assessment and Penetration Testing
PRIMARY TARGET	DVWA running in local lab environment
TESTING ENVIRONMENT	Kali Linux attack machine with Docker-based vulnerable target
PREPARED BY	Jagriti Banerjee
DOCUMENT TYPE	Individual Web VAPT Attack-Section PDF

AUTHORIZED LAB TESTING ONLY

1. Document Purpose

This individual PDF documents the Web VAPT attack section titled 'Vulnerability Scanning and Fingerprinting'. It is designed for the Enterprise Security Assessment Lab GitHub portfolio and describes the objective, tooling, commands, sanitized output, evidence mapping, security interpretation, remediation, and redaction requirements for this specific section.

This document is written as a public portfolio version. It does not contain live client data, production system details, unredacted credentials, session cookies, raw hash dumps, or destructive attack instructions.

Field	Details
Project Area	Web Application Vulnerability Assessment and Penetration Testing
Primary Target	DVWA running in a local lab environment
Testing Environment	Kali Linux attack machine with Docker-based vulnerable target
Testing Style	Reconnaissance -> Enumeration -> Manual validation -> Exploitation proof -> Evidence capture
Methodology Reference	OWASP Web Security Testing methodology, OWASP Top 10, and PTES-style workflow
Prepared By	Jagriti Banerjee

2. Section Overview

Item	Details
Section Number	04
Section Title	Web VAPT Section 04 - Vulnerability Scanning and Fingerprinting
Risk / Severity Style	Informational to Medium depending confirmed issue
OWASP Mapping	Maps primarily to A05 Security Misconfiguration and A06 Vulnerable and Outdated Components.
Primary Evidence Count	5

2.1 Objectives

- Identify common web server misconfigurations.
- Review exposed headers and stack fingerprinting.
- Use scanner output as leads for manual validation.
- Document security posture without overclaiming scanner results.

2.2 Tools Used

Tool	Purpose / Use in this section
Nikto	Web server misconfiguration scanning.
WhatWeb	Technology fingerprinting.
curl	HTTP header and endpoint testing.
Browser	Manual verification and proof screenshots.

3. Methodology and Execution Workflow

The execution workflow follows a controlled lab approach: confirm authorization and target scope, run only the tools required for the section, save outputs, validate observations manually where applicable, capture screenshots, redact sensitive values, and document findings without exaggeration.

3.1 Command Reference

```
# Nikto web scan
nikto -h http://127.0.0.1:8081 -o tool-outputs/nikto-scan-findings.txt

# WhatWeb technology fingerprinting
whatweb http://127.0.0.1:8081

# Header review
curl -I http://127.0.0.1:8081
```

3.2 Expected / Sanitized Output

Nikto produced scan findings for the lab target.
WhatWeb identified web technologies and server details.
Findings were reviewed as leads, not blindly reported as confirmed vulnerabilities.

3.3 Validation Criteria

- The target must be the local DVWA or authorized lab environment only.
- The output must match the intended section objective.
- Evidence screenshots must clearly show the validation result or tool output.
- Any sensitive values must be blurred, cropped, removed, or replaced with placeholders.
- The section must not claim findings that are not supported by evidence.

4. Evidence Mapping

Evidence File	Evidence Type / Reason
07-nikto-scan-findings.png	Nikto scan findings evidence.
08-whatweb-tech-fingerprint-p1.png	WhatWeb technology fingerprinting evidence.
08-whatweb-tech-fingerprint-p2.png	WhatWeb technology fingerprinting evidence.
08-whatweb-tech-fingerprint-p3.png	WhatWeb technology fingerprinting evidence.
08-whatweb-tech-fingerprint-p4.png	WhatWeb technology fingerprinting evidence.

5. Professional Security Analysis

Automated scanners are useful for coverage, but professional reporting requires interpretation. A scanner finding should be confirmed or framed as informational unless manually validated.

Nikto and WhatWeb are especially useful for missing headers, server version disclosure, default files, and debug-page discovery.

For GitHub, this section demonstrates that the assessment includes both automated and manual techniques.

5.1 Why This Section Matters

This section matters because professional VAPT is not only about running a tool. It requires a clear objective, controlled execution, evidence capture, correct interpretation, and practical remediation. For recruiters and reviewers, this PDF shows that the work was structured, documented, and aligned with real security methodology.

5.2 Common Mistakes to Avoid

- Uploading raw outputs that expose cookies, hashes, paths, or credentials.
- Claiming production-level impact without production context.
- Reporting scanner output as a confirmed vulnerability without validation.
- Using unclear screenshots that do not show the relevant result.
- Mixing unrelated phases in one evidence file without explanation.

6. Risk Analysis

- Missing headers can increase clickjacking, MIME sniffing, or XSS risk.
- Debug pages may disclose environment details.
- Server banners can help attackers select known exploits.

6.1 Real-World Impact Statement

The real-world impact depends on the affected asset, authentication context, data sensitivity, privilege level, exposed functionality, and compensating controls. Because this project uses DVWA in a lab, severity is described as a real-world context estimate rather than a formal client CVSS score.

7. Remediation and Hardening Guidance

- Add security headers such as X-Frame-Options or CSP where appropriate.
- Remove debug pages and default files.
- Reduce server/version disclosure.
- Patch web server and application components.
- Validate scanner results manually before final reporting.

7.1 Verification After Remediation

- 1 Re-run the original test case in the lab or development environment.
- 2 Confirm that the previous vulnerable behaviour no longer appears.
- 3 Check server logs for blocked or rejected attempts.
- 4 Capture new evidence showing the fix or defensive control.
- 5 Update the report status from validated/open to fixed/retested if applicable.

8. GitHub Redaction and Publishing Checklist

Cookies, hashes, credentials, session IDs, shell paths, database output, and private details must be redacted before public upload.

Item	Action Before Upload
PHPSESSID / Cookies	Blur or replace with <REDACTED_COOKIE>.
Password hashes	Blur or replace with <REDACTED_HASH>.
Cracked passwords	Do not publish; replace with <REDACTED_PASSWORD>.
Database dumps	Crop, blur, or summarize instead of uploading raw data.
Webshell paths/files	Do not upload shell source files; redact paths if needed.
Local usernames/paths	Blur if private or unnecessary.
Tool raw outputs	Upload only sanitized outputs or summary documents.

8.1 Safe Git Commands Before Push

```
git status
git diff --cached
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"
```

9. Conclusion

This document completes the individual Web VAPT PDF for Section 04. It documents the section objective, execution workflow, evidence mapping, professional interpretation, risk, remediation, and GitHub safety checks. The content is aligned to the actual Enterprise Security Assessment Lab Web VAPT evidence set and avoids unsupported production claims.

Legal and ethical notice: This documentation is for authorized lab testing only. Do not apply these techniques to public or third-party systems without explicit written permission.